

Other Common Operators

ELEC1006: ENGINEERING COMPUTING

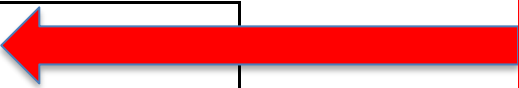
Operators

- Assignment (=)
- Arithmetic operators (+, -, *, /, %)
- Compound assignment (+=, -=, *=, /=, %=, >>=, <<=, &=, ^=, |=)
- Increase and decrease (++ , --)
- Relational and equality operators (<, >, <=, >=, ==, !=)
- Logical operators (!, &&, ||)
- Bitwise operators (&, |, ^, ~, <<, >>)
- Conditional operator (?)
- Comma operator (,)
- Bitwise operator
- Explicit type casting operator
- Precedence of operators.

Relational and Equality Operators

Used to evaluate a comparison between two expressions

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to



== is not the same as =

e.g. a=2, b=3, c=6;

```
(a == 5)      // evaluates to false since a is not equal to 5.  
(a*b >= c)    // evaluates to true since (2*3 >= 6) is true.  
(b+4 > a*c)   // evaluates to false since (3+4 > 2*6) is false.  
((b=2) == a)  // evaluates to true.
```

What will this statement produce `b=2==a` ????

Logical Operators

! Is the logic **NOT** operation (only has ONE operand)

&& is the logic **AND** operation (needs TWO operand)

|| is the logic **OR** operation (Two operand)

eg. `a=3, b=5;`

`!(a==b)` is true

(similar to `a!=b`)

`(a>3)&&(b<7)` is false

`(a>3)|| (b<7)` is true

Conditional Operator

- Evaluates an expression returning a value if that expression is true and a different one if the expression is false.

- Its format is:

condition ? result1 : result2

True – result 1

False – result 2

`7==5 ? 4 : 3`

`7==5+2 ? 4 : 3`

`5>3 ? a : b`

`a>b ? a : b`

```
#include <iostream>
using namespace std;

int main ()
{
    int a,b,c;

    a=2;
    b=7;
    c = (a>b) ? a : b;

    cout << c;

    return 0;
}
```

Comma Operator

- Used to separate two or more expressions that are included where only one expression is expected
- When the set of expressions has to be evaluated for a value, only the rightmost expression is considered:

`a = (b = 2, b+3);`  `b=2; a=b+3;`

Bitwise Operator

- Single bit operations.
- 1 bit only has binary (2) states.
 - 0 (FALSE) or 1 (TRUE).
- NOT inverts states, i.e. $1 \rightarrow 0$ and $0 \rightarrow 1$.
- Refer to the Truth Table for the possible states.
- SHL refers to shift left operation:
 - $001 \rightarrow 010$.
- SHR refers to shift right operation:
 - $110 \rightarrow 011$.

operator	asm equivalent
&	AND
	OR
^	XOR
~	NOT
<<	SHL
>>	SHR

Truth Table

Input 1	Input 2	AND	OR	XOR
0	0	0	1	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Type Casting Operator

- Operations are performed between operands of the same type.
- If not of the same type, C++ will convert one to be the type of the other
- This can impact the results of calculations.

e.g: float a=10.0;

int b=4;

int c=a/b;



c = 2 instead of c = 2.5 due to c being an integer instead of float.

Precedence of Operators

Level	Precedence group	Operator	Description	Grouping
1	Scope	::	scope qualifier	Left-to-right
2	Postfix (unary)	++ --	postfix increment / decrement	Left-to-right
		()	functional forms	
		[]	subscript	
		. ->	member access	
3	Prefix (unary)	++ --	prefix increment / decrement	Right-to-left
		~ !	bitwise NOT / logical NOT	
		+ -	unary prefix	
		& *	reference / dereference	
		new delete	allocation / deallocation	
		sizeof	parameter pack	
		(type)	C-style type-casting	
4	Pointer-to-member	.* ->*	access pointer	Left-to-right
5	Arithmetic: scaling	* / %	multiply, divide, modulo	Left-to-right
6	Arithmetic: addition	+ -	addition, subtraction	Left-to-right
7	Bitwise shift	<< >>	shift left, shift right	Left-to-right
8	Relational	< > <= >=	comparison operators	Left-to-right
9	Equality	== !=	equality / inequality	Left-to-right
10	And	&	bitwise AND	Left-to-right
11	Exclusive or	^	bitwise XOR	Left-to-right
12	Inclusive or		bitwise OR	Left-to-right
13	Conjunction	&&	logical AND	Left-to-right
14	Disjunction		logical OR	Left-to-right
15	Assignment-level expressions	= *= /= %*= += -=>>= <<= &= ^= =	assignment / compound assignment	Right-to-left
		?:	conditional operator	
16	Sequencing	,	comma separator	Left-to-right

More info on C++

- [1] cplusplus.com: Operators
<https://cplusplus.com/doc/tutorial/operators/>